



2

LECTURE 2

The Big Four + Operator Overloading

CS24 · Fall 2026 · Diba

How to build your arsenal to deal with memory-related bugs

list_demo.cpp: Track down the bug using a debugger

Next: Building your conceptual knowledge of C++ memory management:
the Big Four, via the Complex class.

The Big Four

Four special functions to initialize, copy, overwrite, and tear down an object.

- | | | |
|---|--|--|
| 1 | Constructor | Initialize a new object |
| 2 | Copy constructor | Initialize a new object from an existing one |
| 3 | Copy assignment
(aka. <code>operator=</code>) | Overwrite an existing object |
| 4 | Destructor | Teardown owned resources |

C++ writes all four for you by default. You write your own only when needed

IClicker Poll

Does Complex need teardown code?

- A — Yes, it needs a custom destructor
- B — No, the compiler-generated default is fine
- C — Not sure

ANSWER

No because nothing is owned on the heap.

There's nothing to tear down. Complex is two doubles, no pointers, no heap allocation.

Misconception

The destructor does not delete the object. The language destroys the object when its scope ends, or when you call delete on a pointer to it.

The destructor only cleans up what the object owns.

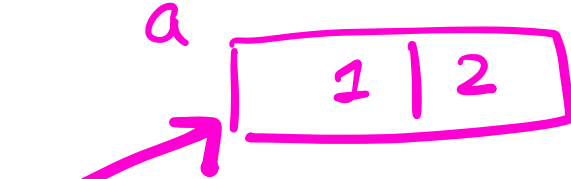
ONE MORE HIDDEN THING

Meet the "this" pointer

Complex `a (1, 2);`
`a.print();`

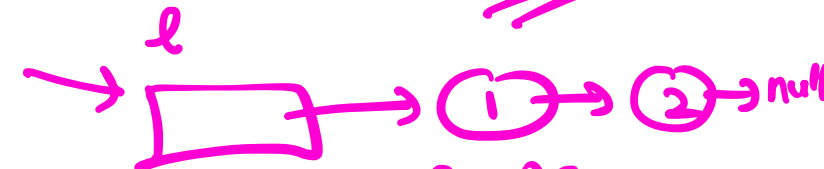
CustomList `l;`
`l.push-back (1);`
`(2)`
`//`

This
hidden, refers to the
implicit object.



The diagram shows a variable 'a' with an arrow pointing to a rectangular box. Inside the box, there are two columns separated by a vertical line, with the number '1' in the left column and the number '2' in the right column.

Heap



The diagram shows a variable 'l' with an arrow pointing to a rectangular box. From the right side of the box, an arrow points to a circle containing the number '1'. From the right side of the circle '1', an arrow points to another circle containing the number '2'. From the right side of the circle '2', an arrow points to the word 'null'.

When `l` goes out of scope.
We can no longer access the nodes
Memory leak!

Another way to break your linked list

Debug using a multi-pronged approach: combination of debugger, code comprehension, conceptual understanding of the big four, and print statement

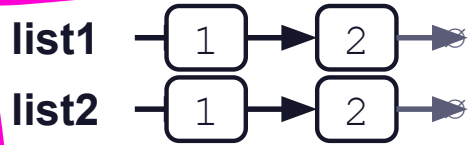
SWITCHING BACK TO CUSTOMLIST

Choose the diagram that shows the default behavior of this code.

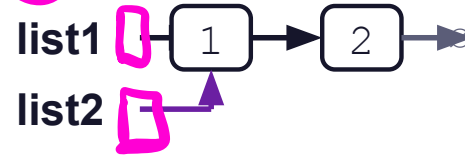
`CustomList list2 = list1;`

CustomList & list2 = list1;

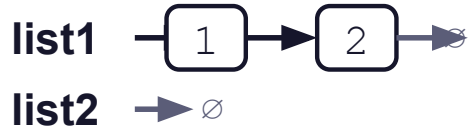
A Two separate copies



B Same nodes as list1



C list2 comes back empty

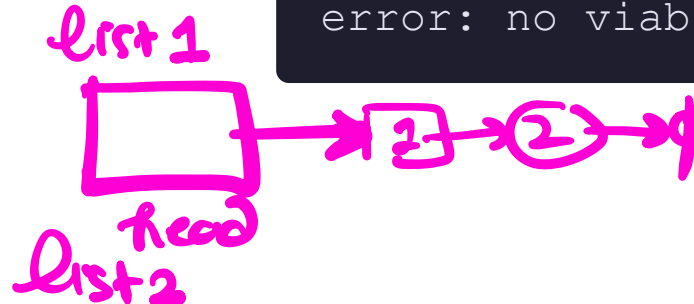


D Doesn't compile

`error: no viable '='`

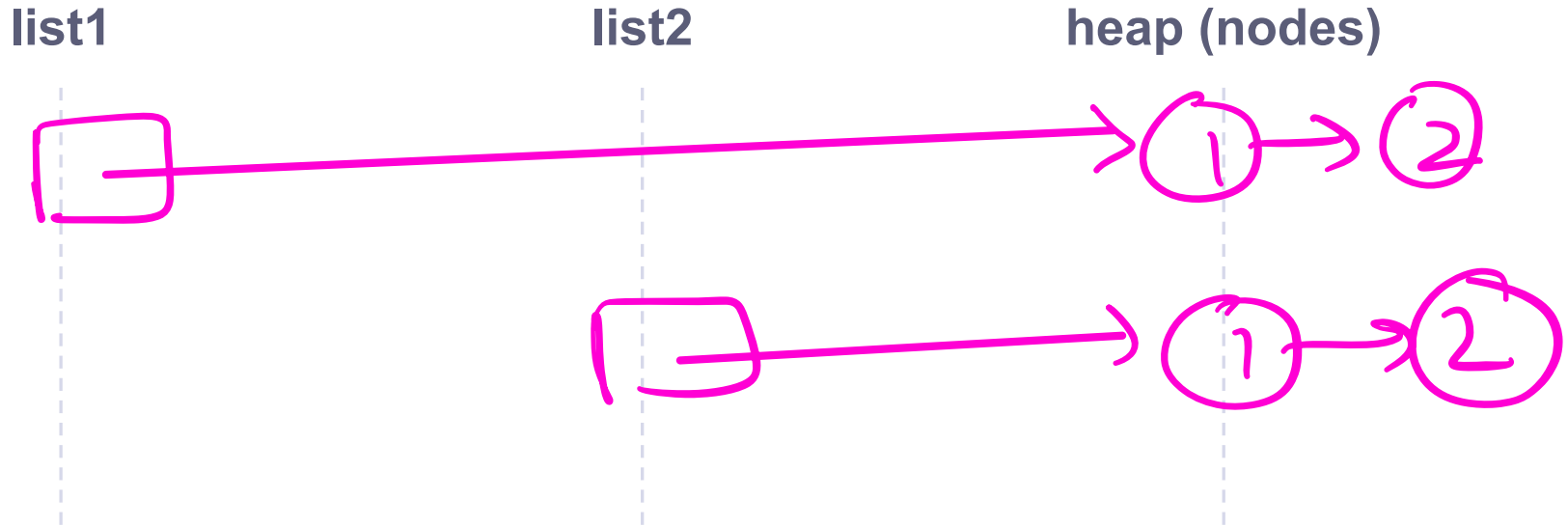
Actually

*Default copy constructor
only copies
list1's head pointer
into list2's head*



Draw the intended behavior of this code (when the copy constructor is correctly implemented)

```
CustomList list2 = list1;
```

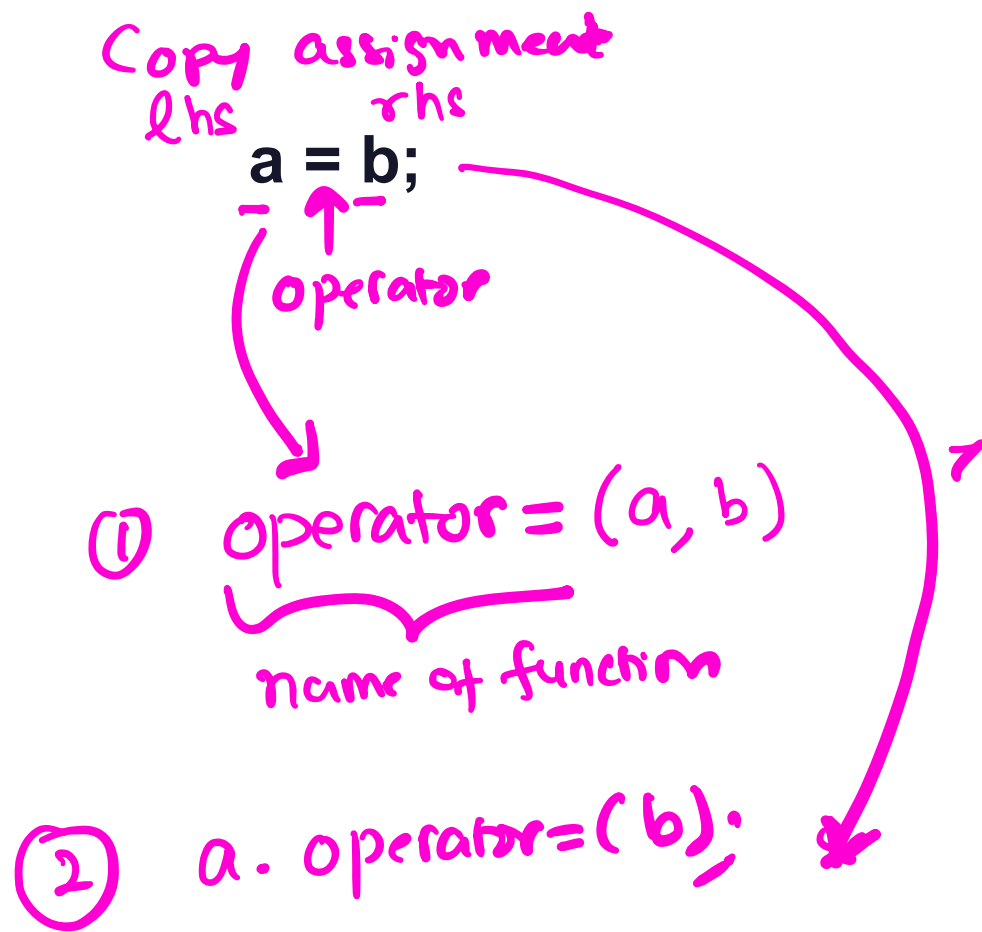


THE RULE OF THREE

Write one of the three, you need all three.

Destructor · Copy constructor · Copy assignment (operator=)

Recall: Operators applied on class objects are function calls!



No default!

`a = b + c;`

WATCH THIS BREAK

cout << **c1;**
ostream operator Complex

```
cout << c1;
```

```
error: no match for 'operator<<'
```

```
(operand types are 'ostream' and 'Complex')
```

A third break today — this one isn't a memory bug. The compiler has no idea how to print a Complex type object.

IClicker POLL

lhs. cout << c1; is really a call to one of these

A

```
cout.operator<<(c1);
```

member of ostream

B

```
operator<<(cout, c1);
```

free function

Which one can we actually write?

Why isn't `operator<<` a member of `Complex`?

```
cout << c1;    // if operator<< were a member of Complex,  
               // compiler will not find a matching function
```

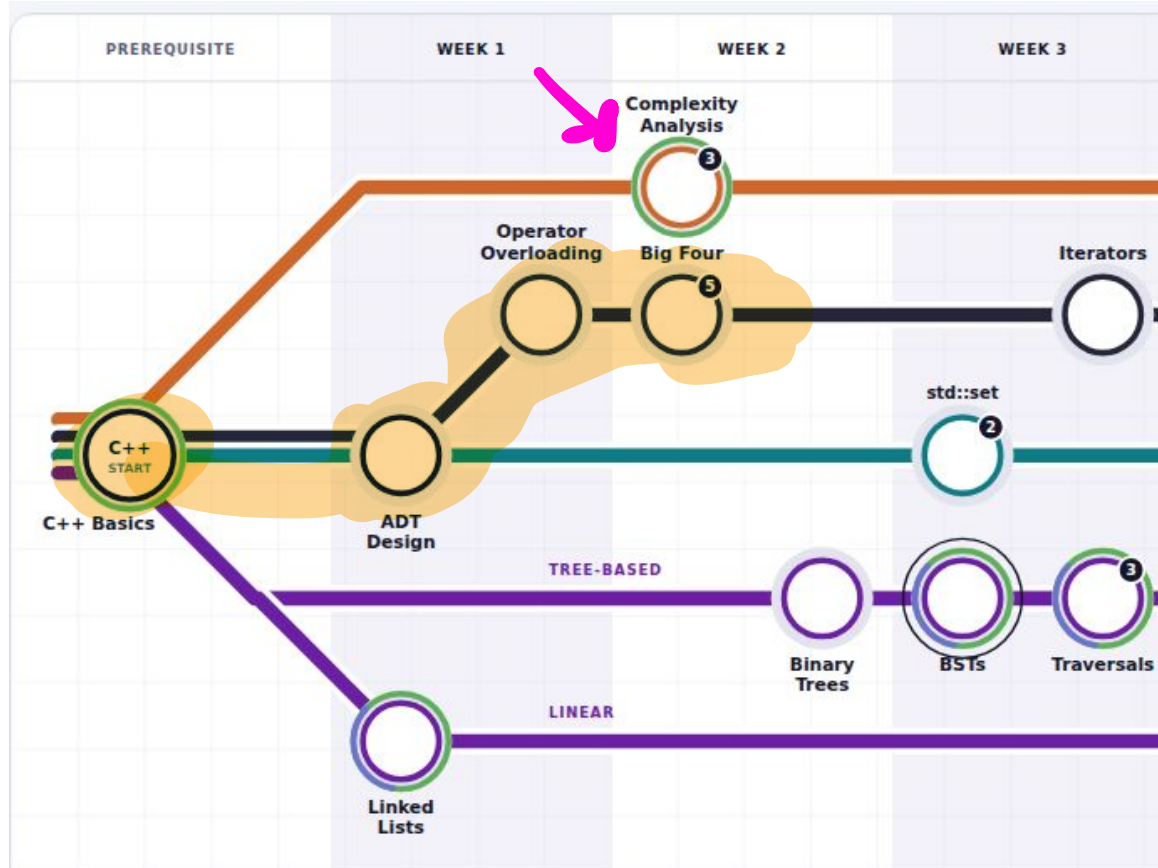
You already saw one overload today — copy assignment, a member function. `operator<<` can't be a member: the left-hand side of `cout << c1` is an ostream, not a `Complex`. So it has to be a free function instead.

The signature

```
ostream& operator<<(ostream& out, const Complex& c);
```

- ostream& on the left, const Complex& on the right
- Returns ostream& so chaining works: cout << a << b is (cout << a) << b
- ostream objects can't be copied, so it's passed and returned by reference, never by value

Back to the map



TODAY'S TAKEAWAYS

- Two memory bugs: a dangling pointer, a double free
- One operator overloading gap: `cout << c1` didn't compile
- All three were fixed by understanding the Big Four and operator overloading

BEFORE NEXT TIME

Read and analyze your code

- Read: Das Gupta Chapter 0 
- Analyze the running time (Big-O) of your iterative and recursive solutions to the reverse linked list function. 
